

Lab 3 – Odometry based Navigation

By now you have worked with ROS and got some basic knowledge. In this Lab, you will learn how to control robot movements, and how to record odometry and velocity data for creating trajectory and velocity graphs. Note that only odometry data is used in this Lab.

3.1 Control the robot movement

Task 1: You should control your robot to move forward, go through the 1st gap and the 2nd gap, and reach to the charger position, as shown in Figure 4.1.

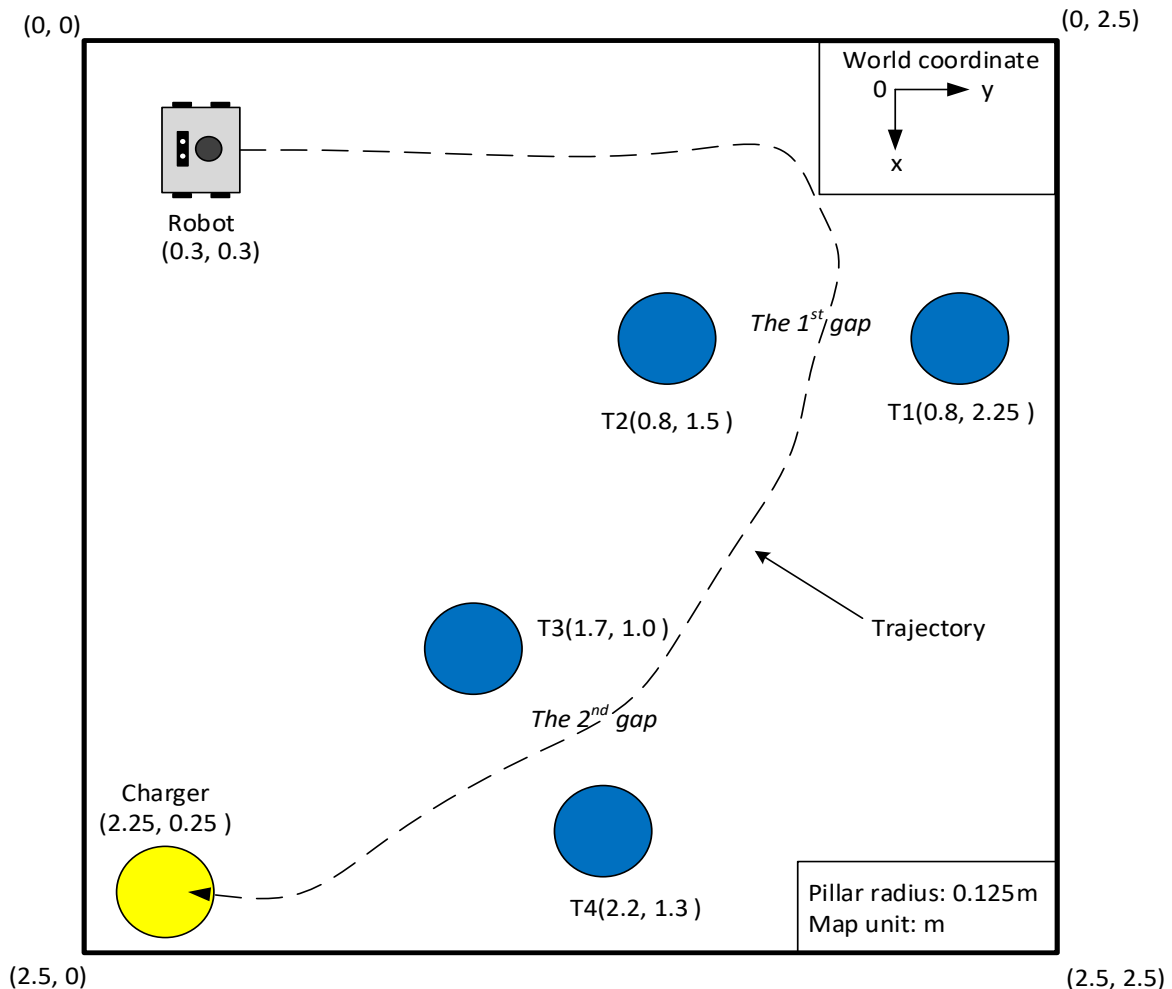


Figure 4.1 The map for the mobile robot to navigate.

In the 1st stage, you will control your robot to move along a straight line to the position (0.3, 1.1) and then gradually turn right to reach the position (0.8, 1.875), i.e., the middle between pillars T1 and T2.

Step 1

Use the following commands to open [tutorial_pkg_node.cpp](#)

```
cd ~/M-Drive/ros2_ws/src/tutorial_pkg/src
gedit tutorial_pkg_node.cpp
```

Then add the yellow-highlighted code below:

```

#include <chrono>
#include <functional>
#include <memory>
#include <string>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"
#include "geometry_msgs/msg/twist.hpp"
#include "geometry_msgs/msg/pose.hpp"
#include "nav_msgs/msg/odometry.hpp"

using namespace std::chrono_literals;
using namespace std;

class Stopper : public rclcpp::Node{
public:
    /* velocity control variables*/
    constexpr const static double FORWARD_SPEED_LOW = 0.1;
    constexpr const static double FORWARD_SPEED_MIDDLE = 0.3;
    constexpr const static double FORWARD_SPEED_HIGH = 0.5;
    constexpr const static double FORWARD_SPEED_STOP = 0;
    constexpr const static double TURN_LEFT_SPEED_LOW = 0.3;
    constexpr const static double TURN_LEFT_SPEED_MIDDLE = 0.6;
    constexpr const static double TURN_LEFT_SPEED_HIGH = 1.0;
    constexpr const static double TURN_RIGHT_SPEED_LOW = -0.3;
    constexpr const static double TURN_RIGHT_SPEED_MIDDLE = -0.6;
    constexpr const static double TURN_RIGHT_SPEED_HIGH = -1.0;

    /* class constructor */
    Stopper():Node("Stopper"), count_(0){
        publisher_ = this->create_publisher<geometry_msgs::msg::Twist>("cmd_vel", 10);

        odomSub_ = this->create_subscription<nav_msgs::msg::Odometry>("odom", 10, std::bind(&Stopper::odomCallback, this, std::placeholders::_1));
    };

    /* moving function */
    void startMoving();
    void moveStop();
    void moveForward(double forwardSpeed);
    void moveRight(double turn_right_speed);
    void moveForwardRight(double forwardSpeed, double turn_right_speed);
    void odomCallback(const nav_msgs::msg::Odometry::SharedPtr odomMsg);
    double PositionX=0.3, PositionY=0.3, homeX=0.3, homeY=0.3;
    double odom_landmark1=1.38, odom_landmark2=0.80;
    int stage=1;

private:
    // Publisher to the robot's velocity command topic
    rclcpp::Publisher<geometry_msgs::msg::Twist>::SharedPtr publisher_;
    rclcpp::TimerBase::SharedPtr timer_;
    rclcpp::Subscription<nav_msgs::msg::Odometry>::SharedPtr odomSub_;
    size_t count_;
};

void Stopper::moveStop(){
    auto msg = geometry_msgs::msg::Twist();
    msg.linear.x = FORWARD_SPEED_STOP;
    publisher_ ->publish(msg);
}

```

```

void Stopper::moveForward(double forwardSpeed){
    //The default constructor to set all commands to 0
    auto msg=geometry_msgs::msg::Twist();
    //Drive forward at a given speed along the x-axis.
    msg.linear.x = forwardSpeed;
    publisher_ ->publish(msg);
}

void Stopper::moveRight(double turn_right_speed){
    auto msg = geometry_msgs::msg::Twist();
    msg.angular.z = turn_right_speed;
    publisher_ ->publish(msg);
}

void Stopper::moveForwardRight(double forwardSpeed, double turn_right_speed){
    auto msg = geometry_msgs::msg::Twist();
    msg.linear.x = forwardSpeed;
    msg.angular.z = turn_right_speed;
    publisher_ ->publish(msg);
}

void Stopper::odomCallback(const nav_msgs::msg::Odometry::SharedPtr odomMsg){
    PositionX = odomMsg->pose.pose.position.x + homeX;
    PositionY = odomMsg->pose.pose.position.y + homeY;
    RCLCPP_INFO(this->get_logger(),"Robot Postion: %.2f, %.2f",PositionX, PositionY );
    RCLCPP_INFO(this->get_logger(), "Robot stage: %d ", stage );
    switch(stage){
        case 1:
            if (PositionY < odom_landmark1)
                moveForward(FORWARD_SPEED_MIDDLE);
            else stage = 2;
            break;
        case 2:
            if (PositionX < odom_landmark2)
                moveForwardRight(FORWARD_SPEED_MIDDLE,TURN_RIGHT_SPEED_MIDDLE);
            else stage = 6;
            break;
        case 6:
            moveStop();
            break;
    }
}

void Stopper::startMoving(){
    RCLCPP_INFO(this->get_logger(), "Start moving");
    while (rclcpp::ok()){
        rclcpp::WallRate loop_rate(20);
        // Remove the lines highlighted in red
        moveForward(FORWARD_SPEED_LOW);
        RCLCPP_INFO(this->get_logger(), "Robot speed: %f", FORWARD_SPEED_LOW);
        auto node = std::make_shared<Stopper>();
        rclcpp::spin_some(node);
        rclcpp::spin(node);
        loop_rate.sleep();
    }
}

int main(int argc, char *argv[]){
    rclcpp::init(argc, argv);
    Stopper stopper;
    stopper.startMoving();
    return 0;
}

```

Step 2

To add the dependency into your code, you should go to the following directory

```
cd ~/M-Drive/ros2_ws/src/tutorial_pkg
```

then use the following command to open the *package.xml* file

```
gedit package.xml
```

add the following below the line `<depend>geometry_msgs</depend>`

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>tutorial_pkg</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="jr18744@todo.todo">jr18744</maintainer>
  <license>TODO: License declaration</license>

  <buildtool_depend>ament_cmake</buildtool_depend>
  <depend>rclcpp</depend>
  <depend>std_msgs</depend>
  <depend>geometry_msgs</depend>
  <depend>nav_msgs</depend>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

Use the following command to open the file *CMakeLists.txt*

```
gedit CMakeLists.txt
```

then add the following line below `find_package(geometry_msgs REQUIRED)`

```
# find dependencies
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(nav_msgs REQUIRED)
```

Add `nav_msgs` into the following command:

Save the file and exit.

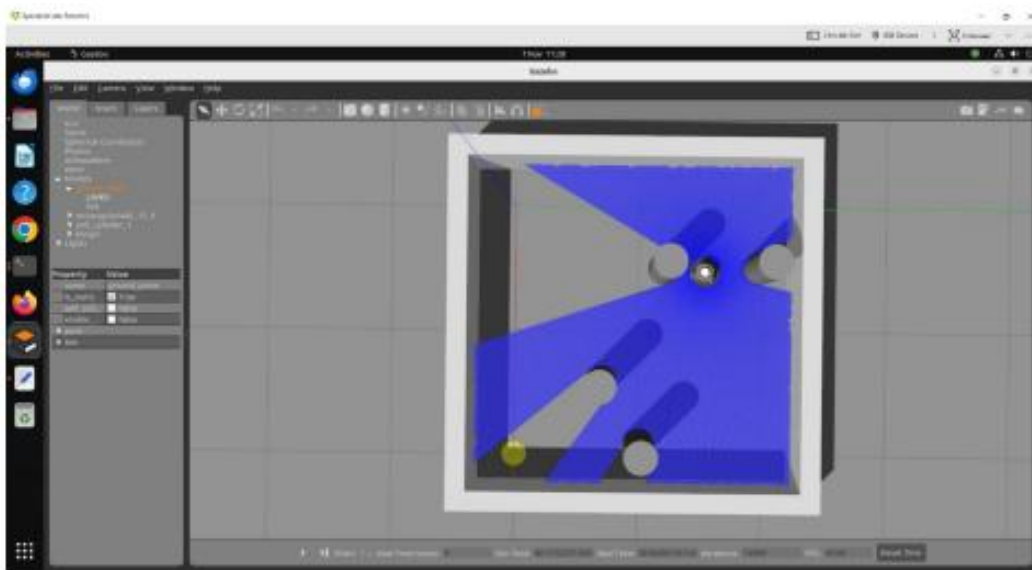
```
add_executable(tutorial_pkg_node src/tutorial_pkg_node.cpp)
ament_target_dependencies(tutorial_pkg_node rclcpp std_msgs geometry_msgs nav_msgs)
```

Step 3: To compile and run your code.

```
cd ~/M-Drive/ros2_ws
source install/setup.sh
export TURTLEBOT3_MODEL=burger
colcon build
ros2 launch tutorial_pkg tutorial_turtlebot.launch.py
```

Step 4: As is shown in the figure below.

The robot first travels along a straight-line path, then turns toward the middle of Gap 1 and finally stops in the middle between T1 and T2.



Step 5: To enable the robot to continue its movement until reaching the charger, you should

1) Using the following commands to *open tutorial_pkg_node.cpp*

```
cd ~/M-Drive/ros2_ws/src/tutorial_pkg/src
gedit tutorial_pkg_node.cpp
```

2) Add the highlighted yellow line below, and select the suitable values for *odom_landmark3*, *odom_landmark4* and *odom_landmark5* so the robot can go through Gap 2 and reach the charger position.

```
double odom_landmark1=1.38, odom_landmark2=0.80;
double odom_landmark3=?, odom_landmark4=?, odom_landmark5=?;
int stage=1;
```

3) Add the highlighted yellow code below:

```

void Stopper::odomCallback(const nav_msgs::msg::Odometry::SharedPtr odomMsg){
    PositionX = odomMsg->pose.pose.position.x + homeX;
    PositionY = odomMsg->pose.pose.position.y + homeY;
    RCLCPP_INFO(this->get_logger(),"Robot Postion: %.2f, %.2f",PositionX, PositionY);
    RCLCPP_INFO(this->get_logger(),"Robot stage: %d ", stage );
    switch(stage){
        case 1:
            if (PositionY < odom_landmark1)
                moveForward(FORWARD_SPEED_MIDDLE);
            else stage = 2;
            break;
        case 2:
            if (PositionX < odom_landmark2)
                moveForwardRight(FORWARD_SPEED_MIDDLE,TURN_RIGHT_SPEED_MIDDLE);
            else stage = 6;
            break;
        case 3:
            if(PositionX < odom_landmark3)
                moveForward(FORWARD_SPEED_HIGH);
            else stage = 4;
            break;
        case 4:
            if(PositionX < odom_landmark4)
                moveForwardRight(FORWARD_SPEED_MIDDLE, TURN_RIGHT_SPEED_MIDDLE);
            else stage = 5;
            break;
        case 5:
            if(PositionY > odom_landmark5)
                moveForward(FORWARD_SPEED_LOW);
            else stage = 6;
            break;
        case 6:
            moveStop();
            break;
    }
}

```

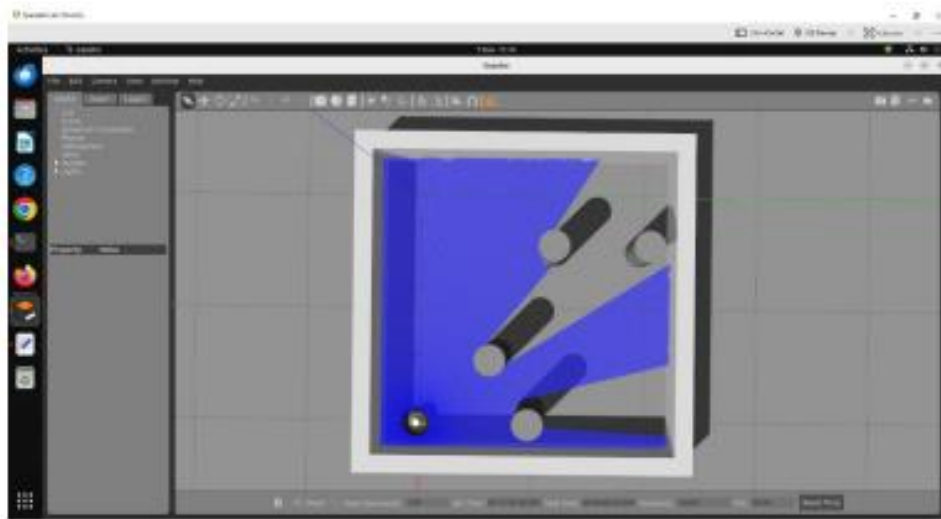
Step 6: Then use the following commands to compile and run your program :

```

cd ~/M-Drive/ros2_ws
source install/setup.sh
export TURTLEBOT3_MODEL=burger
colcon build
ros2 launch tutorial_pkg tutorial_turtlebot.launch.py

```

Step 7: You should see the robot reaching the charger position as shown below after running.



3.2 Odometry data for creating trajectory graph

In the above code, a subscriber is created for receiving odometry messages being published to “/odom” topic. To check or monitor the messages on “/odom” topic, you can use one terminal to run your code, and another terminal to view the message by using the following command:

ros2 topic echo /odom

You will see the message information for “/odom” topic.

```
- 0.0
- 0.001
---
header:
  stamp:
    sec: 64371
    nanosec: 789000000
  frame_id: odom
child_frame_id: base_link
pose:
  pose:
    position:
      x: 2.0270063743492552
      y: 0.18375042694528926
      z: 0.019976925505120893
    orientation:
      x: 2.1275248221770983e-05
      y: -1.2183885126810611e-05
      z: -0.5308587447543839
      w: 0.8474603191399094
  covariance:
    - 1.0e-05
    - 0.0
    - 0.0
    - 0.0
    - 0.0
    - 0.0
    - 0.0
    - 1.0e-05
    - 0.0
    - 0.0
    - 0.0
    - 0.0
    - 0.0
    - 0.0
    - 1000000000000.0
    - 0.0
    - 0.0
    - 0.0
    - 0.0
    - 0.0
    - 1000000000000.0
    - 0.0
    - 0.0
    - 0.0
    - 0.0
    - 0.0
    - 1000000000000.0
    - 0.0
    - 0.0
    - 0.0
    - 0.0
    - 0.0
    - 0.001
twist:
```

Task 2 Run your code and record your robot's trajectory in a csv file. Then you should use LibreOffice in Ubuntu or Excel in Windows to plot this trajectory starting from the home position and ending at the charger position. The following code is required to complete this task.

Step 1: Use the following commands to open `tutorial_pkg_node.cpp`

```
cd ~/M-Drive/ros2_ws/src/tutorial_pkg/src
gedit tutorial_pkg_node.cpp
```

Then add the yellow-highlighted code:

```
#include <chrono>
#include <functional>
#include <memory>
#include <string>
#include <fstream>
#include <time.h>
#include <iomanip>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"
#include "geometry_msgs/msg/twist.hpp"
#include "geometry_msgs/msg/pose.hpp"
#include "nav_msgs/msg/odometry.hpp"
std::ofstream robotTrajFile;
```

`robotTrajFile` is a file object we've declared to allow us to record odometry data

Step 2: Add yellow-highlighted code to `void Stopper::odomCallback`

```
void Stopper::odomCallback(const nav_msgs::msg::Odometry::SharedPtr odomMsg) {
    PositionX = odomMsg->pose.pose.position.x + homeX;
    PositionY = odomMsg->pose.pose.position.y + homeY;
    RCLCPP_INFO(this->get_logger(), "RobotPosition: %.2f , %.2f", PositionX, PositionY);
    RCLCPP_INFO(this->get_logger(), "Robot stage: %d ", stage);
    robotTrajFile << PositionX << " " << PositionY << endl;
    switch(stage) {
```

Step 3: Add the yellow-highlighted code to `void Stopper::startMoving`:

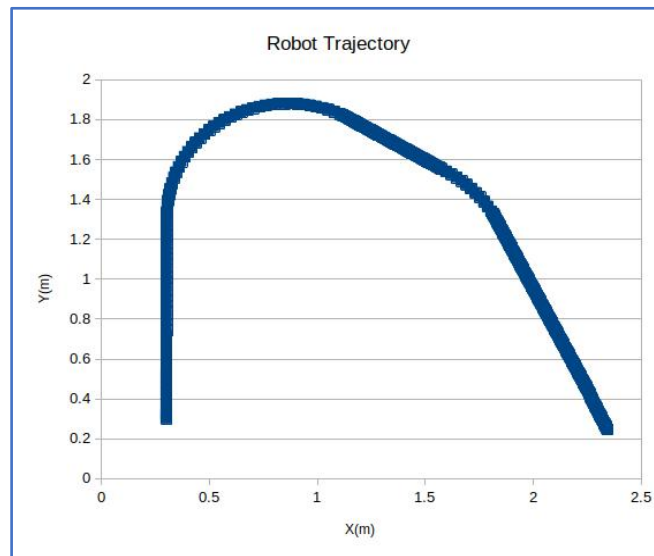
```
void Stopper::startMoving() {
    RCLCPP_INFO(this->get_logger(), "Start moving");
    robotTrajFile.open("~/M-Drive/ros2_ws/src/tutorial_pkg/robotTrajData.csv", ios::trunc);
    while (rclcpp::ok()) {
        rclcpp::WallRate loop_rate(20);
        auto node = std::make_shared<Stopper>();
        rclcpp::spin(node);
        loop_rate.sleep();
    }
    robotTrajFile.close();
}
```

Replace '*' with the path to your workspace, find this with the 'pwd' command

Step 4: Then use the following commands to compile and run your program :

```
cd ~/M-Drive/ros2_ws
source install/setup.sh
export TURTLEBOT3_MODEL=burger
colcon build
ros2 launch tutorial_pkg tutorial_turtlebot.launch.py
```

Step 5: After running your code, you will be able to plot the data in `robotTrajData.csv` using LibreOffice in Ubuntu. It should look like the picture below.



3.3 Velocity graph

The velocity of the robot is also published to the `'odom'` topic, e.g. `odomMsg->twist.twist.linear.x` is the velocity along the forward direction.

Note: `odomMsg` is the odometry message name defined by yourself.

Task 3 Run your code and record your robot's velocity in a csv file. Then you should use Excel to plot a Speed-Time graph. The robot should start from the home position and stop at the charger position. Explain why the velocity graph is in such a condition.

Step 1: Use the following commands to open `tutorial_pkg_node.cpp`

```
cd ~/M-Drive/ros2_ws/src/tutorial_pkg/src
gedit tutorial_pkg_node.cpp
```

You should include the following code below the line `std::ofstream robotTrajFile`

```
std::ofstream robotVelFile;
```

This declares a file object to record velocity data.

Step 2: You should add the following highlighted code into `class Stopper : {}`, in the section containing the velocity control variables.

```
double robVelocity;
```

```
int numberOfCycles=0;
```

Step 3: Add the following highlighted code to `void Stopper::odomCallback()`

```
robVelocity = odomMsg->twist.twist.linear.x;
```

```
robotVelFile << numberOfCycle++ << " " << robVelocity << endl;
```

Step 4: Add the following code to `void Stopper::startMoving`.

```
robotVelFile.open("~/M-Drive/ros2_ws/src/tutorial_pkg/robotTrajData.csv",ios::trunc);
while () {
```

```
    // existing code
```

```
}
```

```
robotVelFile.close();
```

For `robotVelFile.open()`, use the same path you used for `robotTrajFile.open()`

Step 5: Compile the new code and run it using the following commands.

```
cd ~/M-Drive/ros2_ws
```

```
source install/setup.sh
```

```
export TURTLEBOT3_MODEL=burger
```

```
colcon build
```

```
ros2 launch tutorial_pkg tutorial_turtlebot.launch.py
```

Step 6: After running your code, you can plot the data in `robotVelFile.csv` using LibreOffice in Ubuntu. It should look like the picture below.

